Jihwan Choi

Seoul · jhwan333@gmail.com · 010-9692-6526 · jihwan-choi · jhwanchoi · jhwanchoi.github.io

Profile

Software engineer building and operating **production Generative AI** and the MLOps inference/evaluation platform behind autonomous-driving AI. I ship a **cost-aware multimodal VLM evaluation system** (rules → local VLM → AWS Bedrock escalation), LLM cost/usage observability, RAG, and agentic tooling — on top of a shared Kubernetes GPU cluster I co-designed (scheduler & queue policy) and the **SVEvalFlow** workflow I own end to end (Inferit, MTBF Validation, PRISM). I turn engineering into measurable business value: a 20,000-hour validation cut from 6+ months to 1.5 months (430,851 clips at 99.87%), a cost-driven on-premise move saving ₩448M (55%) vs AWS, and a log-analysis platform over 80M daily events preventing ~₩100M/month in overbilling.

Experience

Stradvision, Software Engineer — Seoul, South Korea — July 2022 – Present

MLOps Platform [Link]

Shared GPU Cluster Co-Design & SVEvalFlow (Inference/Eval) Owner *(Oct 2025 – Present)*

Tech: RKE2, Ray, KubeRay, KServe, Knative, MLflow, Airflow, ArgoCD, Harbor, lakeFS, GitHub Actions, Helm

- Co-designed and built a shared RKE2 Kubernetes cluster (60+ heterogeneous GPUs), **leading scheduler & queue policy design**. Shipped an application-level scheduler that preempts only the GPU deficit from running bulk jobs for urgent evaluations and automatically restores capacity; integrated 15+ MLOps components
- Collaborate on the training workflow (SVDeepFlow) and **solely own the inference/evaluation workflow (SVEvalFlow)** — Inferit, MTBF, and PRISM below
- Built 3 CI/CD pipelines (GitHub Actions + ArgoCD) and Scale-to-Zero serverless inference (KServe/Knative)
- Drove internal adoption of AI-native development tooling (Claude Code, Gemini CLI, Codex; MCP/Plugin/Skill) through hands-on engineering sessions and n8n automation prototypes

Inferit [Link]

svnet3 Build/Inference Acceleration & Artifact Management

Tech: FastAPI, Airflow, Kubernetes, lakeFS, Docker, PostgreSQL, HCP Object Storage

- Built svnet3 version/config/artifact management and integrated an MTBF API that resolves or builds worker images by branch/config/commit/CUDA toolchain (cu118/cu128), tracking base, version, digest, and custom overrides
- Accelerated single-machine inference into distributed/parallel execution across cluster GPUs — cut a 20,000-hour validation from an estimated 6+ months to 1.5 months
- Versioned customer option files (e.g., camera parameters), removing the 1–2 days of engineer-PM search and hand-off each ticket previously required
- Supports 5+ major release versions, with per-ticket automated inference across OD/LD/TSR/TLR features (run multiple times a week) and integrated Evaluation-Driven Development (eval automation)

MTBF Validation [Link]

Large-Scale svnet3 Reliability Validation

Tech: FastAPI, Airflow, Kubernetes (RKE2/NKS), PostgreSQL, HCP/NCP Object Storage, AWS Bedrock

- Automated svnet3 inference on customer-ODD data before/after each release to flag false positives and AEB events. Processed 430,851 clips at 99.87% success (15,233h), with 205% higher throughput and 67.3% lower processing time vs a single machine — ~10 real false positives across 15,233h (~1 per 5,000h)
- Proposed and executed a cost-driven Cloud to On-Premise transition: AWS ₩820M to On-Premise ₩370M (55% saved); automated the pipeline for single-person operation
- Migrated the full stack to **NCP NKS/L40S, Object Storage, and NCR** via GitOps. Processed 57,745 commands over 1,000h in 23h 57m (99.94%; zero infrastructure-attributable failures), autoscaling GPU nodes 1→14 within 13 minutes and back to 1
- Operate 7-type validation and Visual Inspection with **rules → local VLM → AWS Bedrock escalation**; only human-

approved results enter final KPIs

PRISM [Link]

Customer Jira-Integrated ISP Conversion & Re-simulation Pipeline *(Nov 2025 –)*

Tech: FastAPI, Airflow, PostgreSQL, AWS (SQS, Lambda), Terraform

- **Phase 1 (pre-deployment):** Event-driven ISP conversion via Jira Webhook to SQS to Lambda, integrated with the in-house ISP tool. Connectivity test complete, with full deployment to follow Inferit. Designed for ~150 requests/month, ~80% faster than manual
- **Phase 2 (Resim, in dev):** Built the initial version and now act as technical lead, guiding another org's developer on the implementation. Dual-version inference (issue + latest) via SWIT Docker Images, Airflow DAG trigger

Data & Labeling Automation *(2022 – 2025)*

- **WLS:** Analyzed 80M daily labeling-log events to prevent ~₩100M/month overbilling (settlement-audit confirmed). Pinpointed bottlenecks feeding UI/UX and the ALAS model (+20% labeling efficiency) (Django, MongoDB, AWS SQS)
- **ALAS:** Auto-labeling microservice, reduced manual annotation workload by ~35% (FastAPI, AWS SQS, AWS SNS, AWS ECS/EKS, CDK)
- **IGS:** External-customer data ingestion service planning & architecture design (design-only) (Django, Airflow, Kubernetes)
- **SURF Compass:** Device monitoring service planning & design (design-only) (FastAPI, PostgreSQL)
- **RNS:** Automated release-note generation, 50% faster authoring (4h to 2h) (NestJS, Atlassian API)
- **SLT:** 2D LD labeling tool, improved productivity by 30% (C++/MFC)

Korea Institute of Robotics & Technology Convergence, Researcher — Pohang, South Korea — Jan 2022 – July 2022

- Developed real-time gesture-based Human-Robot Interaction system
- Supported autism diagnosis assistance robot project

Skills

Languages: Python, C++, TypeScript
Backend: Django, FastAPI, NestJS
Database: PostgreSQL, MongoDB, Redis
Cloud: AWS (EC2, EKS, ECS, SQS, SNS, Lambda, S3, Bedrock), NCP (NKS, Object Storage, NCR), GCP (Vertex AI)
Infrastructure: Kubernetes (RKE2, NKS), Docker, Terraform, CDK, Rancher
ML/MLOps: Ray, KubeRay, MLflow, KServe, Knative, lakeFS
Workflow: Airflow, Celery
CI/CD: ArgoCD, GitHub Actions, Harbor, Helm
Monitoring: Grafana, Prometheus, Loki, DataDog
Generative AI / LLM: Local VLM, AWS Bedrock (Claude), Vertex AI (Gemini), multimodal, structured output, RAG / embeddings, prompt engineering, LLM cost & observability, agentic / tool-use, Claude Code / MCP / Skills
Tools: Git, Jira, Confluence, Atlassian API

Generative AI Engineering

- **Tiered multimodal VLM evaluation (production):** Evolved an initial Gemini/Vertex AI service using a 1,110-case golden set: rules handle 79.5%, local VLM evaluates the remainder, and only ~16% of ambiguous issue-axis cases escalate to AWS Bedrock Claude Opus. Added self-consistency, JSON-schema output, transfer approval, and human review; unreviewed results are excluded from KPIs (MTBF)
- **LLM cost & usage observability (AWS Bedrock):** Token-level, per-model/per-team cost-per-request attribution via CloudWatch + Slack, Terraform IaC. 90%+ accuracy vs billing, adopted as an org cost-optimization tool
- **RAG knowledge system (LLM Wiki):** Hybrid keyword + embedding retrieval (text-embedding-3-small, cosine top-k) → grounded answers with citations, freshness, and uncertainty. CI-rebuilt index (Git + Markdown)
- **Agentic LLM tooling:** NL intent → cluster diagnostics/storage tools (Claude Code Skills), multi-step orchestration + guardrails (dry-run, rollback, gates). Multi-model self-critique across Claude/GPT/Gemini (jhwanchoi/dotfiles)
- **Tech:** Local VLM, AWS Bedrock (Claude), Vertex AI (Gemini), embeddings/RAG, Claude Code, Gemini CLI, Codex, MCP, CloudWatch, Terraform, n8n

Education

BS **Handong Global University**, Electronic Engineering & Computer Science — GPA 3.3/4.5 Feb 2018 – Feb 2022

Coursework: Data Structure, Algorithm, Computer Architecture, Deep Learning, Machine Learning